

THE BATCH

June 26, 2026

What Matters in AI Right Now

[Subscribe](#) [Submit a tip](#)

Dear friends,

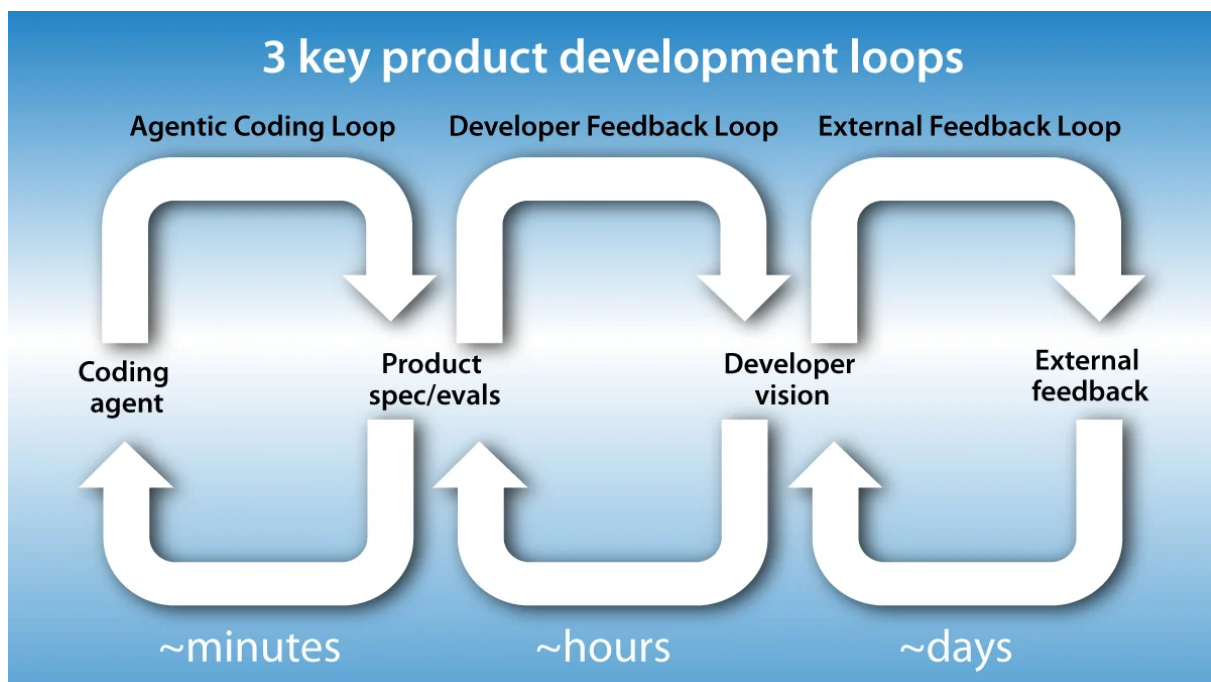
“Loop engineering” is a hot buzzphrase after Boris Cherney (Claude Code’s creator) and Peter Steinberger (OpenClaw’s creator) both mentioned it on social media. Loops are now a key part of how we get AI agents to iterate at length to build software. In this letter, I’d like to share my 3 key loops, shown in the image below, for building o-to-1 products. These loops guide not just how I build software, but also how I decide what software to build.

Agentic coding loop: Given a product specification and optionally a set of evals (that is, a dataset against which to measure performance), we can have an AI agent write code, test its work, and keep iterating until the code is bug-free and meets its specification. This idea of closing the loop took off around the end of last year, and it has been a game changer in enabling coding agents to work longer productively without human intervention. For example, over the weekend, I was building an app for my daughter to practice typing, and my coding agent could easily work for around an hour, using a web browser to check what it had built multiple times before getting back to me, without needing my intervention.

The engineering loop executes quickly. Every few minutes, the coding agent might build and test a new version of the software. I hear frequently from developers who are finding new ways to engineer more effective engineering loops. This is an active area of invention!

Developer feedback loop: In this loop, a developer examines the current product and steers the coding agent to improve it. Last year, a lot of developers (including me) were acting as the QA (quality assurance) function for our coding agents, manually finding bugs and then asking the agent to fix them. But with coding agents much more able to test their own code, the amount of time we need to spend on this function has decreased significantly. This allows us to make higher-level product decisions, such as what key features to offer, where the UI needs improvement, and so on.

The developer-feedback loop operates over time intervals between tens of minutes and hours — that's how frequently a developer might review a product and give feedback. In the case of the typing app, I changed my mind a few times about the visual design, what cat costumes she can unlock as she learns (she loves cats), and the user flow for a grown-up to log in and steer the child's learning experience.



When a developer has a clear vision for what to build, it is still a lot of work to translate that vision into a specification for a coding agent to implement. Further, after the developer has seen an implementation, they might update (or perhaps clarify) the spec to steer it toward what they want. If you find that the system repeatedly runs into certain problems, building a set of evals for the agent becomes useful.

AI-native teams are increasingly using AI to help shape product direction, for example, automating the gathering and analysis of usage data, summarizing written and verbal customer feedback, or carrying out competitive analysis. However, for pretty much all the products I'm involved in, I see humans as having a significant context advantage over current AI systems — we know a lot more than the AI system about the users and the context the product has to operate in — and thus humans play a critical role. Many people describe this human contribution as “taste,” but I prefer to think of it as humans having a context advantage, since that gives us a clearer path to helping AI systems get better. This also speaks to why this step can't be automated: So long as the human knows something the AI does not, human-in-the-loop is needed to inject that knowledge into the system.

External feedback loop: This includes a wide range of [tactics](#) like asking a few friends for feedback, launching to alpha testers, or putting the code into production with A/B testing. These tactics are usually slow, rarely taking less than hours and sometimes taking days or even weeks. This data informs the [developer vision](#), which in turn continues to drive the detailed product spec, which in turn drives the coding agent.

With coding agents speeding up software development, more engineers are starting to play a partial product management role. For many engineers who are growing into this role, the hardest part is shaping the product vision and striking a balance between building (bridging the gap between vision and spec) and getting user feedback to evolve the vision. It is important to do both!

I will write more about how to do this in future letters, but for now, I find it encouraging that engineers are playing an expanded role (just as product managers and designers now do more engineering).

Keep building!

Andrew